

User Guide: Documentation Agent Extension

Generated from USER-GUIDE.md

Artifact type: other

Complete guide for using the AI-powered Documentation Agent to generate professional documentation from your markdown files.

Table of contents

User Guide: Documentation Agent Extension

What This Extension Does

Prerequisites

Quick Start (5 Minutes)

1. Start the Backend

Windows

Linux/Mac

2. Configure Extension

3. Start Automatic Processing (Recommended)

Windows (from backend repo)

Or from Spec Kit

4. Create Markdown → Get Professional PDF!

Extension Commands

Core Commands

``/speckit.ext.setup``

``/speckit.ext.docgen``

``/speckit.ext.status``

``/speckit.ext.regenerate``

``/speckit.ext.watch``

``/speckit.ext.toggle-mode``

Processing Methods

Method 1: Automatic File Watching (■ Recommended)

Start watcher

Method 2: Manual On-Demand

Method 3: Git Commit Hooks

Understanding AI Features

AI-Powered Transformation

spec

api

db

Auto-Generated Diagrams

Architecture Diagrams

System Architecture

API Diagrams

API Endpoints

Data Model Diagrams

Data Model

Sequence Diagrams

Authentication Flow

State Diagrams

Order States

Document Enhancements

Executive Summary

Table of Contents

Glossary

Cross-References

Enhanced PDF Structure

What You Get in the PDF

Viewing Generated PDFs

Option 1: Copy from Docker

List available PDFs

Copy specific PDF

Option 2: Mount Local Directory

Option 3: Database Query

Complete Workflow Examples

Example 1: Create New Feature Spec

1. Start watcher (if not running)
2. Create spec file

User Authentication System

Overview

Architecture

API Endpoints

Data Model

Security

3. Save file → Auto-processed!

4. Check status

Run in Spec Kit: `/speckit.ext.status`

5. Get PDF

Example 2: Batch Process Multiple Files

1. Switch to fast mode

Run in Spec Kit: `/speckit.ext.toggle-mode`

Choose: [2] Rule-Based ■

2. Create multiple files

3. Let watcher process all (fast!)

4. Switch back to AI mode

Run in Spec Kit: `/speckit.ext.toggle-mode`

Choose: [1] AI-Powered ■

5. Regenerate important ones for quality

`/speckit.ext.regenerate spec-1.md`

Example 3: Draft → Final Workflow

Phase 1: Draft (fast, rule-based)

1. Toggle to rule-based
2. Write multiple draft specs

Edit, save, iterate quickly

Phase 2: Review

3. Review draft PDFs

Phase 3: Final (quality, AI-powered)

4. Toggle to AI mode
5. Regenerate final versions
6. Get professional PDFs with diagrams

Tips for Best Results

Writing for AI

User Authentication Feature Specification

System Architecture

API Endpoints

spec

Best Practices

Troubleshooting

Backend Not Running

No Diagrams Generated

System Architecture

API Endpoints

Watcher Not Detecting Changes

Restart watcher

Poor Quality Output

Slow Processing

FAQ

Q: Do I need an OpenAI API key?

Q: Can I use it offline?

Q: How much does it cost?

Q: What if I don't have AI?

Q: Can I customize diagram styles?

Q: Where are PDFs stored?

Q: Can I share the backend with my team?

Q: Does it work with other IDEs?

Command Quick Reference

What Gets Processed?

Body

User Guide: Documentation Agent Extension

Complete guide for using the AI-powered Documentation Agent to generate professional documentation from your markdown files.

What This Extension Does

The Documentation Agent transforms simple markdown files into **professional PDF documentation** with:

- **AI-Powered Intelligence** - Professional titles and abstracts (not just "spec.md") - Context-aware content analysis - Smart section classification
- **Auto-Generated Diagrams** - Architecture diagrams from component descriptions - API endpoint visualizations - Data model ER diagrams - Workflow/sequence diagrams - State machine diagrams
- **Document Enhancements** - Executive summaries for specs and plans - Automatic table of contents with page numbers - Glossary extraction for technical terms - Cross-references between sections - Intelligent section splitting (no more walls of text) - Best practices recommendations
- **Flexible Processing** - Automatic file watching (process on save) - Manual on-demand generation - Git commit hooks - Toggle between AI and rule-based modes ---

Prerequisites

1. **Spec Kit or Specify** (version 0.11.9+) installed 2. **Docker Desktop** running on your machine 3. **Extension installed** in your workspace at ``.specify/extensions/docs-agent/`` ---

Quick Start (5 Minutes)

1. Start the Backend

```
```powershell
```

#### ***Windows***

```
cd path\to\speckit-docs-agent .\start.ps1
```

#### ***Linux/Mac***

```
cd path/to/speckit-docs-agent ./start.sh ``` Wait for: `■ Services are running!`
```

#### ***2. Configure Extension***

In your workspace, run: ```` /speckit.ext.setup ```` Enter: - **API URL**: ``http://localhost:8000`` - **API Key**: ``dev-key``

#### ***3. Start Automatic Processing (Recommended)***

```
```powershell
```

Windows (from backend repo)

```
.\start-watcher.ps1 -WorkspaceRoot "C:\path\to\your\workspace"
```

Or from Spec Kit

```
/speckit.ext.watch ```
```

4. Create Markdown → Get Professional PDF!

Create any `.md` file, save it, and within 2-3 seconds you'll have: - Professional PDF with diagrams - Executive summary - Table of contents - Glossary - Auto-detected architecture/API/data diagrams ---

Extension Commands

Core Commands

`/speckit.ext.setup`

****Configure backend connection (run once per workspace)**** ``` /speckit.ext.setup ``` Prompts for: - API Base URL (default: `http://localhost:8000`) - API Key (default: `dev-key`) Saves to: `.specify/extensions/docs-agent/config.yml` ---

`/speckit.ext.docgen`

****Generate PDF from currently open markdown file**** ``` 1. Open a .md file in editor 2. Run: /speckit.ext.docgen 3. Wait 2-5 seconds 4. Check status with /speckit.ext.status ``` ****What happens:**** - Reads active file content - AI analyzes and enhances content - Generates diagrams automatically - Creates professional PDF - Stores in database with version tracking ---

`/speckit.ext.status`

****View all processed documents**** ``` /speckit.ext.status ``` ****Shows:**** ``` Project: my-workspace Artifacts: ✓ specs/feature/spec.md [spec] - rendered (v2) ✓ architecture.md [other] - rendered (v1) ■ plan.md [plan] - stale (needs regeneration) ``` ****Status Icons:**** - ✓ (rendered) - PDF ready - ■ (stale) - Content changed, needs update - ■ (pending) - Not yet processed - ✗ (failed) - Processing error ---

`/speckit.ext.regenerate`

****Force regenerate PDF (bypass deduplication)**** ``` /speckit.ext.regenerate specs/feature/spec.md ``` ****Use when:**** - Template or styling changed - Want to force new

version - Deduplication preventing update ---

`/speckit.ext.watch`

****Start automatic file watching**** `` `/speckit.ext.watch` `` ****What it does:**** - Monitors workspace for `.md` files - Auto-processes on file create/save - Polls every 2 seconds - Runs in foreground (Ctrl+C to stop) ****Tip:**** Run in separate terminal for continuous monitoring ---

`/speckit.ext.toggle-mode`

****Switch between AI-powered and rule-based modes**** `` `/speckit.ext.toggle-mode` ``
****Choose mode:**** ****[1] AI-Powered ■**** (Recommended) - Professional titles & abstracts - Context-aware analysis - Smart diagram detection - 95% accuracy - 2-3 sec/file ****[2] Rule-Based ■**** (Fast) - Pattern matching - Basic title extraction - Simple diagrams - 70% accuracy - <100ms/file ****When to use each:**** - ****AI Mode****: Client-facing docs, final output, professional quality - ****Rule-Based****: Quick drafts, batch processing, offline work ---

Processing Methods

Method 1: Automatic File Watching (■ Recommended)

****Best for:**** Active development, continuous documentation ****Setup:**** `` powershell

Start watcher

.\start-watcher.ps1 -WorkspaceRoot "C:\Users\YourName\workspace" `` ****Workflow:**** 1. Watcher runs in background 2. Create/edit any `.md` file 3. Save file 4. Within 2-3 seconds → PDF generated 5. Check with `/speckit.ext.status` ****Benefits:**** - Hands-free operation - Real-time processing - No commands to remember ---

Method 2: Manual On-Demand

****Best for:**** Specific files, controlled processing ****Workflow:**** 1. Open markdown file in editor 2. Run `/speckit.ext.docgen` 3. Wait for confirmation 4. PDF generated ****Benefits:**** - Full control - Process only what you want - No background processes ---

Method 3: Git Commit Hooks

****Best for:**** Version-controlled documentation ****Workflow:**** 1. Edit markdown files 2. Commit changes: `git commit -m "Update docs"` 3. Hook automatically processes committed `.md` files ****Benefits:**** - Tied to version control - Automatic on commit - Team collaboration friendly ---

Understanding AI Features

AI-Powered Transformation

****What AI Does:**** ****Input (Your Markdown):**** ```markdown

spec

auth system

api

POST /login GET /user

db

User table ``` ****AI Output:**** ``` Title: "User Authentication System Specification" Abstract: "This specification defines a user authentication system with login endpoints and user data management, providing secure access control through RESTful APIs." Sections: - API Endpoints (classified as: api_endpoint) - Database Schema (classified as: data_model) Auto-Generated Diagrams: 1. API Endpoint Diagram 2. Data Model ER Diagram ``` ****Without AI (Rule-Based):**** ``` Title: "spec" Abstract: "auth system" Sections: - api (classified as: normal) - db (classified as: normal) No diagrams generated ``` ---

Auto-Generated Diagrams

The system ****automatically detects**** when to create diagrams:

Architecture Diagrams

****Triggered by:**** Sections mentioning "architecture", "components", "system design"

****Example:**** ```markdown

System Architecture

Frontend (React) communicates with Backend API (FastAPI) which stores data in PostgreSQL database. ``` ****Result:**** Component diagram showing 3-tier architecture

API Diagrams

****Triggered by:**** API endpoints like `GET /users`, `POST /login` ****Example:**** ```markdown

API Endpoints

- GET /api/users - List all users - POST /api/users - Create user - DELETE /api/users/{id} - Remove user ``` ****Result:**** API endpoint visualization with methods and routes

Data Model Diagrams

****Triggered by:**** Mentions of "entities", "tables", "schema", "data model" ****Example:**** ````markdown`

Data Model

User entity with profile info Order entity tracks purchases Payment entity for transactions `````
****Result:**** ER diagram with entities and relationships

Sequence Diagrams

****Triggered by:**** Numbered steps, workflows, processes ****Example:**** ````markdown`

Authentication Flow

1. User submits credentials 2. System validates against database 3. System generates JWT token 4. Token returned to user ````` ****Result:**** Sequence diagram showing interaction flow

State Diagrams

****Triggered by:**** State mentions, lifecycle descriptions ****Example:**** ````markdown`

Order States

Orders go through: Pending → Processing → Confirmed → Shipped → Delivered `````
****Result:**** State machine diagram ---

Document Enhancements

Executive Summary

****Auto-generated for:**** Specs and Plans ****Contains:**** - Document overview - Key metrics (sections, tasks, stories) - Recommendations for improvements

Table of Contents

****Always generated**** ****Features:**** - Hierarchical structure - Page numbers - Clickable links in PDF

Glossary

****Auto-extracts**** technical terms in `backticks` ****Example Input:**** ````markdown` The system uses `JWT` for auth and stores data in `PostgreSQL`. ````` ****Glossary Output:**** - ****JWT****: JSON Web Token used for authentication - ****PostgreSQL****: Relational database management system

Cross-References

****Auto-detects**** when sections reference each other Adds "See also: Section Name" links ---

Enhanced PDF Structure

What You Get in the PDF

```
```
■ Professional PDF Output ■■ ■ Cover Page ■ ■■ Title (AI-enhanced) ■ ■■ Abstract ■
■ ■■ Artifact type badge ■ ■■ ■ Executive Summary (for specs/plans) ■ ■■ Overview ■ ■■
Key Metrics ■ ■■ Recommendations ■ ■■ ■ Table of Contents ■ ■■ All sections listed ■
■ ■■ Page numbers ■ ■■ Clickable links ■ ■■ ■ Main Content ■ ■■ Section 1: Overview ■
■ ■■ Intelligent splitting ■ ■ ■■ Cross-references ■ ■ ■■ ■ Section 2: Architecture ■ ■
■ ■■ Content ■ ■ ■■ [■ Auto-Generated Architecture Diagram] ■ ■ ■ ■■ Section 3: API
Design ■ ■ ■■ Content ■ ■ ■■ [■ Auto-Generated API Diagram] ■ ■ ■ ■■ Section 4: Data
Model ■ ■■ Content ■ ■■ [■ Auto-Generated ER Diagram] ■ ■■ ■ Diagrams Section ■
■ ■■ Figure 1: System Architecture ■ ■■ Figure 2: API Endpoints ■ ■■ Figure 3: Data Model
■ ■■ ■ Glossary ■ ■■ Term 1: Definition ■ ■■ Term 2: Definition ■ ■■ ... ■ ■■ ■ Best
Practices ■■ Recommendation 1 ■■ Recommendation 2 ```

```

## ***Viewing Generated PDFs***

### ***Option 1: Copy from Docker***

```
```bash
```

List available PDFs

```
docker exec infra-backend-1 ls -la /app/pdf-output/
```

Copy specific PDF

```
docker cp infra-backend-1:/app/pdf-output/artifact-1.pdf ./my-doc.pdf ```
```

Option 2: Mount Local Directory

Edit `infra/docker-compose.yml`: ```yaml services: backend: volumes: - ../backend:/app -
../pdf-output:/app/pdf-output # Add this ``` Restart: ```bash docker-compose down
docker-compose up -d ``` Now PDFs appear in `pdf-output/` folder!

Option 3: Database Query

```
```bash docker exec -it infra-db-1 psql -U docsagent -d docsagent ``` ```sql -- View all PDFs
SELECT a.source_path, dv.version_no, dv.pdf_path, dv.generated_at FROM doc_versions
dv JOIN artifacts a ON dv.artifact_id = a.id ORDER BY dv.generated_at DESC; ```

```

## ***Complete Workflow Examples***

## **Example 1: Create New Feature Spec**

```bash

1. Start watcher (if not running)

.\start-watcher.ps1 -WorkspaceRoot "C:\workspace"

2. Create spec file

mkdir specs/user-authentication cat > specs/user-authentication/spec.md << 'EOF'

User Authentication System

Overview

Secure login system with JWT tokens.

Architecture

- Frontend: React application - Backend: FastAPI REST API - Database: PostgreSQL

API Endpoints

- POST /api/auth/login - User login - POST /api/auth/logout - User logout - GET /api/auth/me - Get current user

Data Model

User entity with: - id (UUID) - email (string) - password_hash (string) - created_at (timestamp)

Security

- JWT tokens expire after 1 hour - Passwords hashed with bcrypt - Rate limiting on login endpoint EOF

3. Save file → Auto-processed!

4. Check status

Run in Spec Kit: /speckit.ext.status

5. Get PDF

```
docker cp infra-backend-1:/app/pdf-output/artifact-X.pdf ./user-auth-spec.pdf `` **PDF
Includes:** - Professional title - Executive summary - Architecture diagram - API endpoint
diagram - Data model ER diagram - Security best practices - Glossary (JWT, bcrypt, etc.) ---
```

Example 2: Batch Process Multiple Files

```
``bash
```

1. Switch to fast mode

Run in Spec Kit: /speckit.ext.toggle-mode

Choose: [2] Rule-Based ■

2. Create multiple files

```
for i in {1..10}; do echo "# Spec $\n\nContent..." > spec-$i.md done
```

3. Let watcher process all (fast!)

4. Switch back to AI mode

Run in Spec Kit: /speckit.ext.toggle-mode

Choose: [1] AI-Powered ■

5. Regenerate important ones for quality

```
/speckit.ext.regenerate spec-1.md
```

```
`` ---
```

Example 3: Draft → Final Workflow

```
``bash
```

Phase 1: Draft (fast, rule-based)

1. Toggle to rule-based

/speckit.ext.toggle-mode → [2] Rule-Based

2. Write multiple draft specs

Edit, save, iterate quickly

Phase 2: Review

3. Review draft PDFs

docker cp infra-backend-1:/app/pdf-output/... ./drafts/

Phase 3: Final (quality, AI-powered)

4. Toggle to AI mode

/speckit.ext.toggle-mode → [1] AI-Powered

5. Regenerate final versions

/speckit.ext.regenerate important-spec.md

6. Get professional PDFs with diagrams

docker cp infra-backend-1:/app/pdf-output/... ./final/ ``` ---

Tips for Best Results

Writing for AI

****■ DO:**** ```markdown

User Authentication Feature Specification

System Architecture

The system consists of three main components: - Frontend UI (React) - Backend API (FastAPI) - Database (PostgreSQL)

API Endpoints

POST /api/auth/login - Authenticate user GET /api/auth/me - Get current user profile ``` ****■ DON'T:**** ```markdown

spec

stuff about auth api things db things ```

Best Practices

1. **Use descriptive headings** - "System Architecture" not "arch" 2. **Include context** - Explain what components do 3. **List API endpoints clearly** - Use HTTP methods 4. **Describe data models** - Mention entities and relationships 5. **Number workflow steps** - Enables sequence diagrams 6. **Use backticks for terms** - Enables glossary extraction 7. **Write clear abstracts** - First paragraph matters ---

Troubleshooting

Backend Not Running

Symptom: `/speckit.ext.setup`` fails with connection error **Fix:** `bash cd infra docker-compose ps # Check status docker-compose up -d # Start if not running ````

No Diagrams Generated

Symptom: PDF has no diagrams **Causes:** 1. **AI mode disabled** - Toggle to AI mode: `/speckit.ext.toggle-mode`` → [1] 2. **Content too vague** - Add more specific component/API/data descriptions 3. **Wrong section titles** - Use keywords: "architecture", "API", "data model" **Fix:** Make content more explicit: ````markdown`

System Architecture

Frontend → Backend → Database

API Endpoints

GET /users POST /users ```

Watcher Not Detecting Changes

Symptom: Save file but no processing **Checks:** 1. **Watcher running?** Check terminal 2. **Correct workspace?** Verify path 3. **File saved?** Changes must be written to disk 4. **Wait 2 seconds** - Polling interval **Fix:** ````bash`

Restart watcher

Ctrl+C (stop current) `.\start-watcher.ps1 -WorkspaceRoot "correct\path" ````

Poor Quality Output

Symptom: Basic titles, no context **Cause:** Rule-based mode active **Fix:** `speckit.ext.toggle-mode Choose: [1] AI-Powered /speckit.ext.regenerate your-file.md`

Slow Processing

Symptom: Takes 5-10 seconds per file **Cause:** AI mode with complex documents
Solutions: 1. **Accept it** - Quality takes time 2. **Toggle to rule-based** - For speed 3.
Batch process - Use rule-based, then regenerate important files with AI ---

FAQ

Q: Do I need an OpenAI API key?

A: No! The system uses your IDE's AI (Copilot/Claude/Kiro) by default. No additional cost.

Q: Can I use it offline?

A: Yes, toggle to rule-based mode (`/speckit.ext.toggle-mode` → [2]`). You'll get basic processing without AI.

Q: How much does it cost?

A: Free! Uses your existing IDE AI subscription (Copilot/Claude). No additional charges.

Q: What if I don't have AI?

A: Rule-based mode works great for structured markdown. You just won't get the enhanced titles/abstracts.

Q: Can I customize diagram styles?

A: Currently uses Mermaid.js default styles. Customization coming in future updates.

Q: Where are PDFs stored?

A: In Docker volume `/app/pdf-output/` and PostgreSQL database with metadata and versioning.

Q: Can I share the backend with my team?

A: Yes! Deploy to a server and configure team workspaces to point to it.

Q: Does it work with other IDEs?

A: Works with any IDE that has Copilot, Claude, or similar AI assistants installed. ---

Command Quick Reference

| Command | Purpose | When to Use |
|--|----------------------|------------------------|
| <code>/speckit.ext.setup`</code> | Configure connection | Once per workspace |
| <code>/speckit.ext.docgen`</code> | Generate PDF | Open file, run command |
| <code>/speckit.ext.status`</code> | View all docs | Check what's processed |
| <code>/speckit.ext.regenerate`</code> | Force regenerate | Template changed |
| <code>/speckit.ext.watch`</code> | Auto-processing | Continuous work |
| <code>/speckit.ext.toggle-mode`</code> | AI ↔ Rule-based | Speed vs quality |

What Gets Processed?

| File Type | Classification | Special Features |
|---|----------------|-----------------------------|
| <code>spec.md`</code> , <code>requirements.md`</code> | Spec | Executive summary, diagrams |
| <code>plan.md`</code> , <code>design.md`</code> | Plan | Architecture diagrams |
| <code>tasks.md`</code> | Task | Checkbox tracking |
| <code>constitution.md`</code> | Constitution | Governance docs |
| <code>architecture.md`</code> | Other | Architecture diagrams |
| <code>*.md`</code> in <code>/api/`</code> | Other | API diagrams |
| <code>*.md`</code> in <code>/db/`</code> | Other | Data model diagrams |
| Any <code>.md`</code> file | Other | Standard processing |

--- **You're ready to create professional documentation!** For more details: -

- AGENTIC-PIPELINE.md** - How AI works -
- INTELLIGENT-DOCUMENTATION.md** - Diagram features -
- MODE-TOGGLING-GUIDE.md** - AI vs rule-based -
- WATCHER-QUICKSTART.md** - Automatic processing

source: *USER-GUIDE.md* | commit: *n/a*